Лабораторна робота №1

з дисципліни “Інтелектуальний аналіз даних”
ВИКОРИСТАННЯ ПАКЕТУ POLYANALYST
ДЛЯ АВТОМАТИЧНОГО ПОШУКУ ЗНАНЬ
(нелінійні залежності у вигляді правил)

Мета лабораторної роботи: вивчити концепцію та функціональні можливості сучасних професійних систем Data mining на прикладі пакету Polyanalyst; навчитися застосовувати комплексні пакети дейтамайнінгу для пошуку знань та нелінійних залежностей.

Завдання до лабораторної роботи. Завданням на лабораторну роботу є:

- ознайомитися з архітектурою та модулями системи Polyanalyst – як одного з лідерів на ринку систем дейтамайнінгу;
- на підставі знайденого набору вхідних даних за допомогою модуля **Find Laws** автоматично знайти в даних нелинейні залежності та подати результати у вигляді математичної формули – поліному;
- роздрукувати згенеровані системою звіти та графіки.
- захистити роботу перед викладачем.

Постановка задачі полягає в наступному. Студенти знаходять по наведеним гіперпосиланням - вхідні дані, що містять дві економічну інформацію. В шуканих даних потрібно виділити цільову змінну. Необхідно з використанням PolyAnalyst знайти формулу, що пов’язує ці атрибути.

Стисла характеристика системи PolyAnalyst.

Система PolyAnalyst призначена для автоматичного аналізу числових і текстових даних з метою виявлення в них раніше невідомих, нетривіальних, практично корисних і доступних розумінню закономірностей, необхідних для прийняття оптимальних рішень у бізнесі й в інших галузях людської діяльності.

В даний час PolyAnalyst є однією із самих потужних систем Data Mining у світі, реалізованих для Intel платформ і операційних систем Microsoft Windows. Аналогічні системи Data Mining таких провідних виробників, як IBM (Intelligent Miner, Data Miner), Silicon Graphics (SGI Miner), Integral Solutions (Clementine), SAS Institute (SAS) працюють на середніх і великих машинах і коштують десятки і навіть сотні тисяч доларів.

По своїй природі, PolyAnalyst є клієнт/серверним додатком. Користувач працює з клієнтською програмою PolyAnalyst Workplace **(рис. 1)**. Математичні модулі виділені в серверну частину - PolyAnalyst Knowledge Server. Така архітектура надає природну можливість для масштабування системи: від однокористувацького варіанта до корпоративного рішення з декількома серверами.

Розглядаєма версія PolyAnalyst 4.5 включає 17 математичних модулів, заснованих на різних алгоритмах Data і Text Mining. Алгоритми аналізу даних можна об'єднати в групи по їхньому функціональному призначенню: моделювання, прогнозування, кластеризація, класифікація, текстовий аналіз.

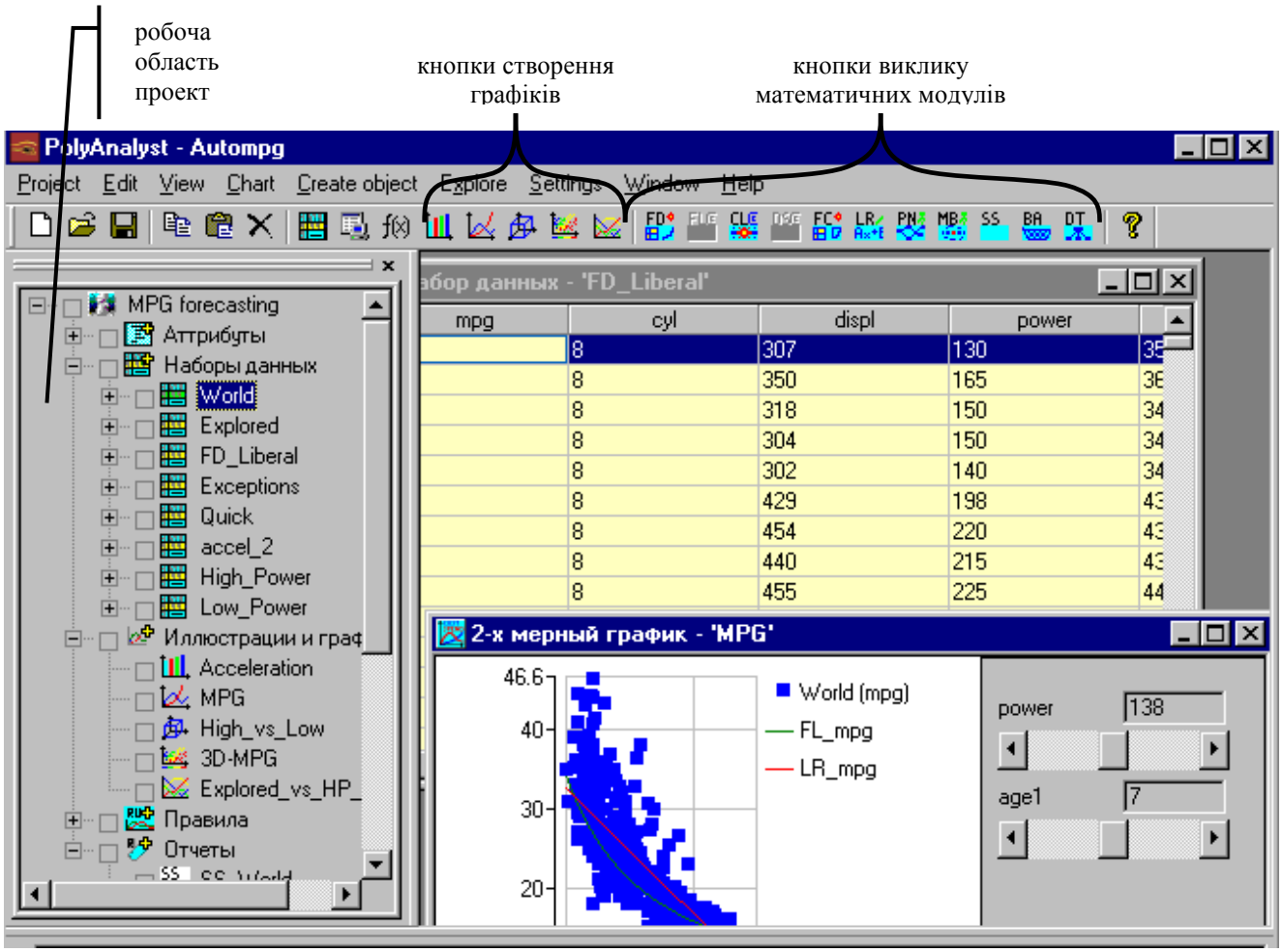


Рис. 1. Загальний вигляд вікна системи Polyanalyst

Ця ЛР, крім повторення основних знань зі статистичного аналізу (що є необхідними для будь-якого проекту Data Science), головним чином, присвячена ознайомленню з роботою алгоритму FL, що є патентом компанії – видавця і розробника пакета Polyanalyst.

Потужний алгоритм Find Laws генерує складні символічні правила, які представляють собою нелінійні залежності в даних. Використовуючи Symbolic Knowledge Acquisition Technology, Find Laws це один з найпотужніших, корисних і унікальних інтелектуального аналізу даних інструментів PolyAnalyst в.

Нижче, викладено набір даних агентства нерухомості, в якому останній стовбчик – ціна – є цільовим атрибутом.

Lab Neuro postanovka3 [Режим совместимости] - Microsoft Excel

KOL_KOMNAT	RAJON	MICRORAJON	PL_ZAG	PL_KOR	PL_KUHN	POVERH	BUDUNOK	SANVUSOL	TEL	PARKET	BALKON	EVROREMONT	KAFEL	ADDITIONAL	GOSTINKA	NEW_BUDUNO	ZENA
1	1	10	36,5	19,0	9,0	0	1	1	1	1	0	0	0	0	0	0	20500
1	1	10	40,0	18,0	12,0	0	1	1	1	1	1	0	0	0	0	0	22000
1	1	17	28,8	14,8	8,5	1	1	1	0	0	0	0	0	0	0	1	13600
1	1	42	38,0	17,7	8,5	1	0	1	1	1	0	0	1	1	0	0	21000
1	1	45	30,0	17,0	6,0	0	1	0	1	0	1	0	0	0	0	0	20000
1	1	45	44,4	19,0	9,1	0	1	0	1	1	1	1	1	0	0	0	36000
1	2	32	28,0	13,5	6,0	1	0	0	0	1	1	0	0	0	1	0	15500
1	2	44	34,0	18,5	7,5	0	0	0	1	0	2	0	0	0	1	0	16500
1	2	44	26,0	13,0	6,0	0	1	0	1	0	2	0	1	0	1	0	15500
1	2	26	22,0	13,0	4,0	1	1	0	1	1	0	0	0	0	0	0	13300
1	2	26	32,0	18,0	6,0	0	1	0	1	1	1	0	0	0	0	0	15500
1	2	26	26,0	13,0	6,0	0	0	0	0	0	0	0	0	0	0	0	15500
1	2	29	49,0	19,0	15,0	1	1	0	0	0	1	0	1	1	0	0	26000
1	2	29	49,0	19,0	14,5	1	1	0	0	0	1	0	0	1	0	1	24000
1	2	29	45,0	17,0	12,0	1	0	1	0	0	1	1	0	0	0	1	23000
1	2	29	44,5	18,0	11,8	1	0	1	0	0	1	1	0	0	0	1	23200
1	2	32	39,0	18,0	9,0	1	0	1	1	1	2	0	1	0	0	0	21500
1	2	32	36,0	18,0	8,5	1	0	1	1	0	2	0	0	0	0	0	22000
1	2	32	41,0	18,0	13,0	1	0	1	0	0	1	0	0	0	0	1	23000
1	2	32	49,0	19,0	14,5	1	1	1	0	0	1	0	0	0	0	1	23500
1	2	32	56,0	20,0	21,0	0	1	1	0	1	2	1	1	1	0	0	34000
1	2	32	56,0	20,0	21,0	0	0	1	1	1	1	0	0	0	0	0	34000
1	2	32	51,0	17,5	17,0	1	1	1	0	0	0	0	0	0	0	1	26000
1	2	32	46,0	18,0	10,2	0	0	0	1	1	2	1	1	0	0	0	23000
1	2	44	30,0	15,0	7,0	0	1	0	0	0	0	0	0	1	0	0	15000
1	2	44	45,0	17,0	13,0	1	0	1	0	0	1	0	0	0	0	1	23000
1	2	44	39,0	19,8	7,5	0	0	1	1	1	0	0	1	1	0	0	21500
1	2	44	45,1	17,2	13,1	1	0	1	0	0	1	0	0	0	0	1	23000

Lab Neuro postanovka3

Укажите ячейку и нажмите ВВОД или выберите "Вставить"

ХІД РОБОТИ:

1. Запуск пакету здійснюється через кнопку **Пуск → Программы → PolyAnalyst 4.4**. В результаті з'явиться діалогове вікно, яке для зручності запропонує користувачу один з основних напрямків роботи з пакетом (рис. 2).

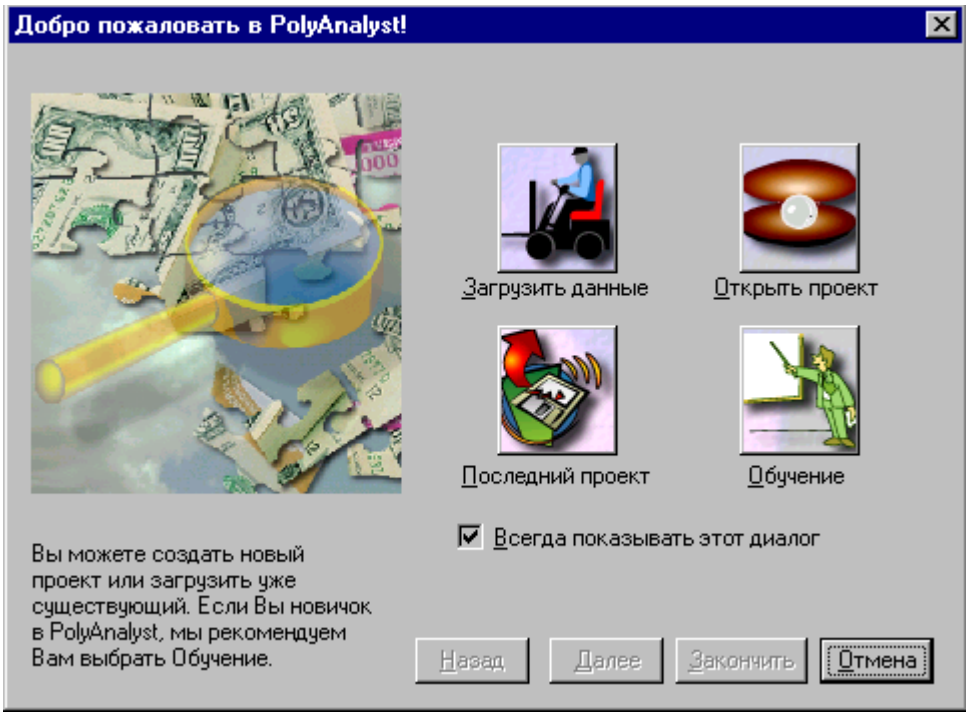


Рис. 2. Діалогове вікно початку роботи з PolyAnalyst

2. Натиснемо кнопку **Отмена**. Після чого відкриється головне вікно програми (рис. 1). Для початку вирішення поставленої задачі необхідно створити файл нового проекту, для цього оберіть пункт головного меню **Project → New**. В результаті запуститься майстер вибору джерела даних (рис. 3). PolyAnalyst дає змогу обрати одне з восьми джерел імпортування даних для подальшого дейтамайнінгу.

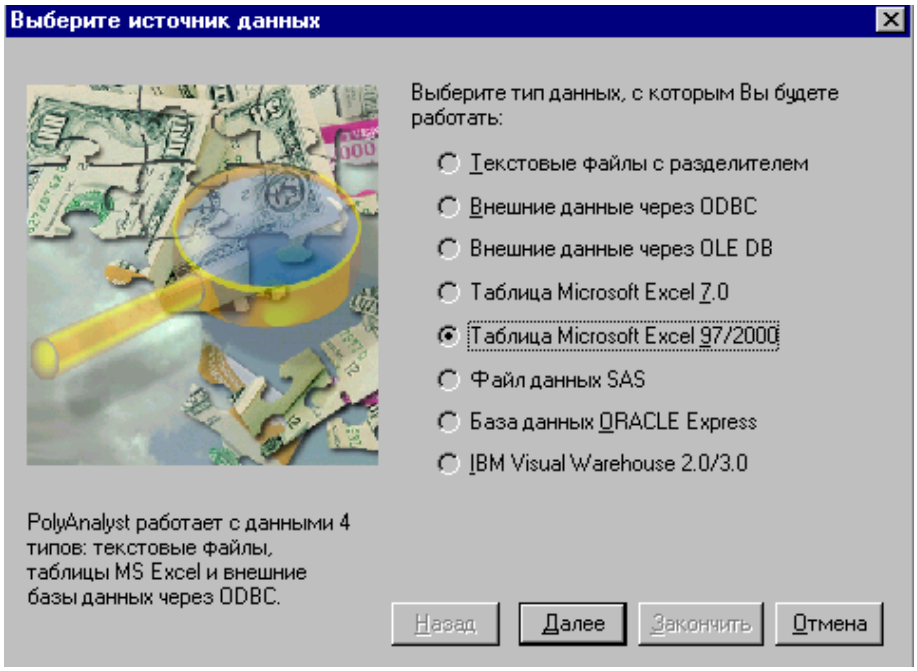


Рис. 3. Вікно вибору джерела даних

Виберемо в якості джерела файл відповідного типу (попередньо відшукавши файл даних зп допомогою гіперпосилань в кінці даної інструкції) та натиснемо кнопку **Далее**. У наступному вікні (рис. 4) натиснемо кнопку **Выбрать** та вкажемо на отриманий від викладача файл-завдання з вхідними даними, та натискаємо **Далее**.

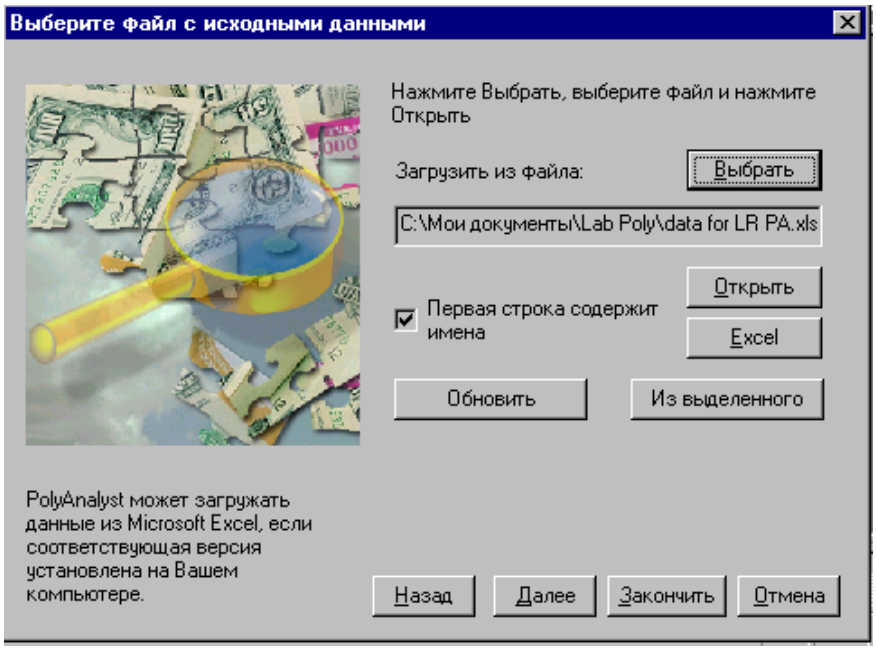


Рис. 4. Вікно вибору файлу даних

На наступних трьох кроках PolyAnalyst пропонує *при необхідності*:

- скоригувати типи даних (рис. 5);
- встановити фільтр на вхідні дані (рис. 6);

- встановити опцію випадкового відбору певної кількості записів із загальної таблиці (рис. 7).

Для даної лабораторної роботи залишаємо за змовчуванням всі вищеперелічені опції. І, нарешті, у вікні під назвою **“Начать загрузку данных”** натискаємо кнопку **Закончить**.

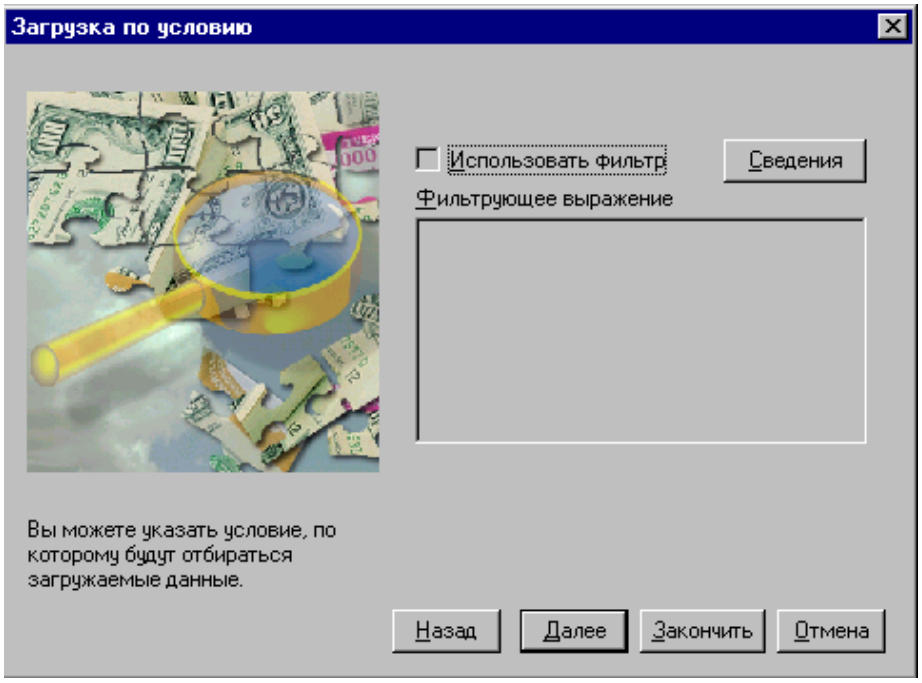


Рис. 5. Діалогове вікно “Загрузка по условию”

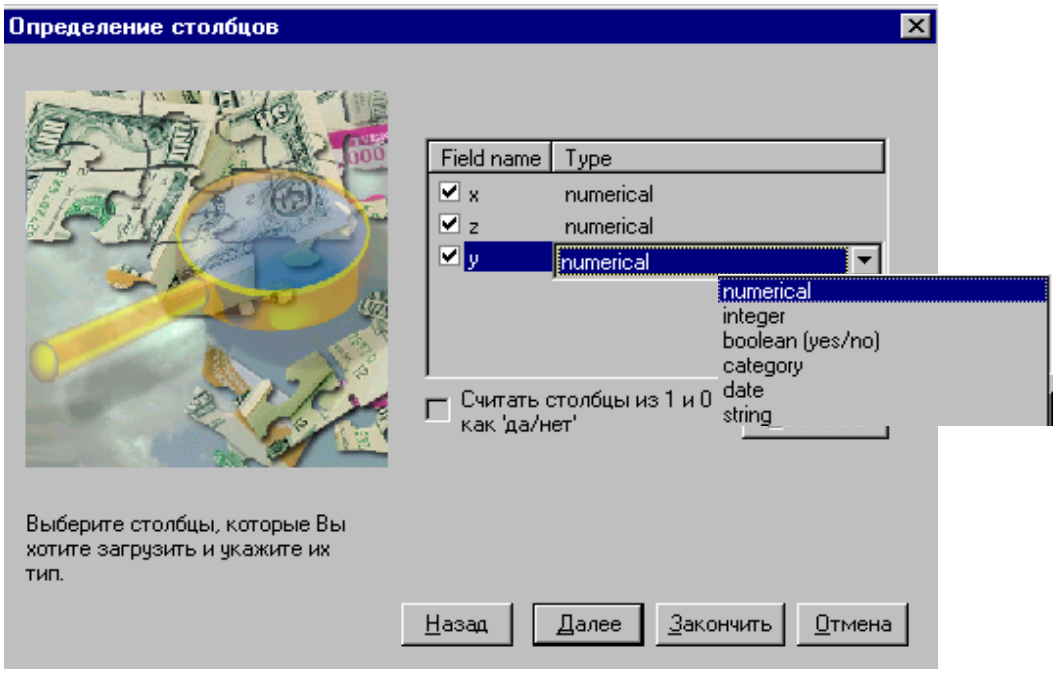


Рис. 6. Діалогове вікно “Определение столбцов”

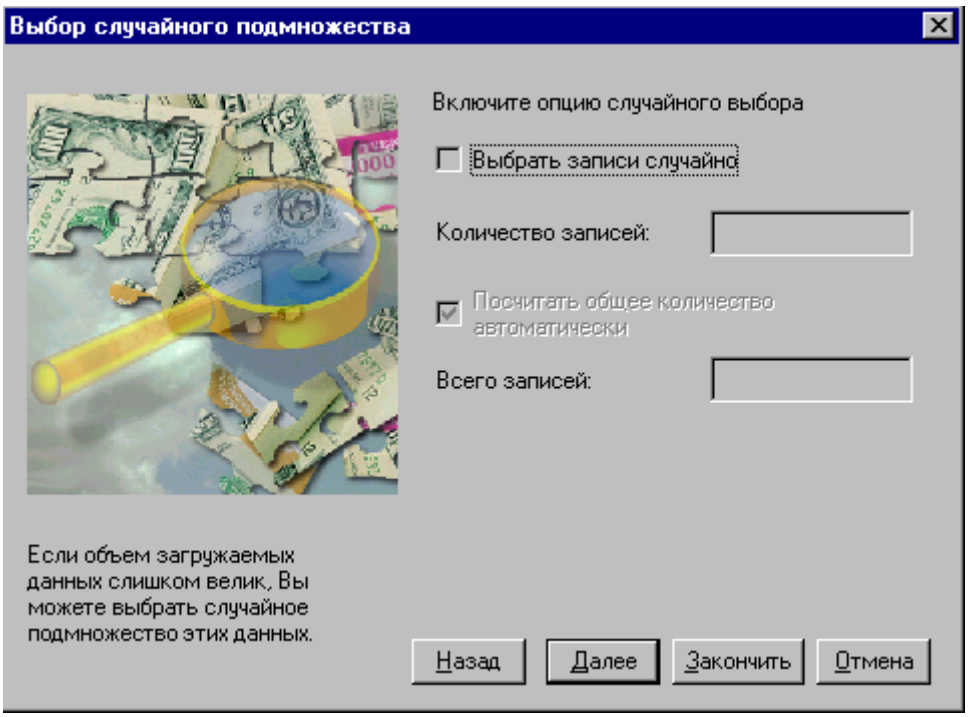


Рис. 7. Діалогове вікно “Выбо случайного подмножества”

3. Після завантаження даних автоматично відкриється вікно для редагування проекту (рис. 8). Після заповнення поля **Project Name** (назва проекту) та, при необхідності, **Field Comment** (коментарів до полів), натискаємо кнопку **OK**.

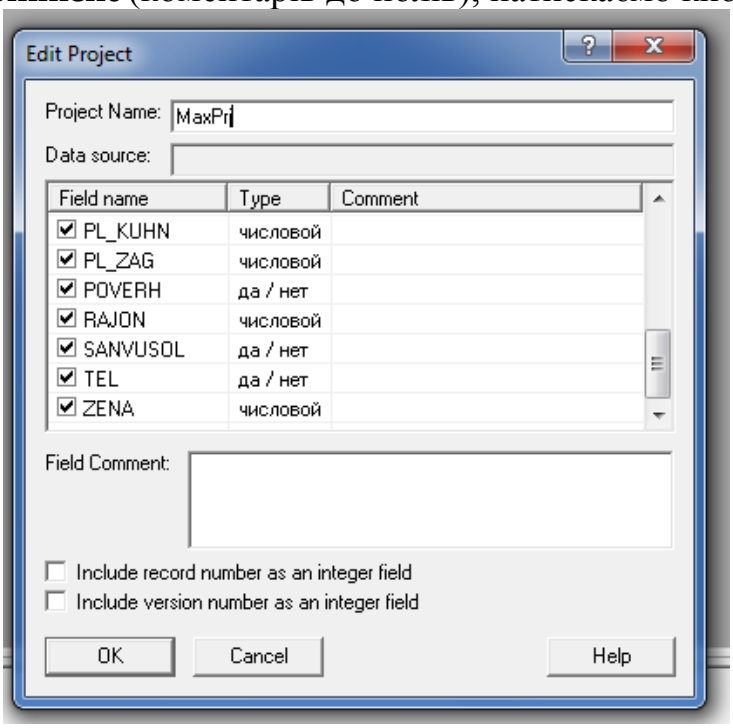


Рис. 8. Вікно редагування проекту

В результаті робочий екран пакету PolyAnalyst приймає вигляд як на рис. 9.

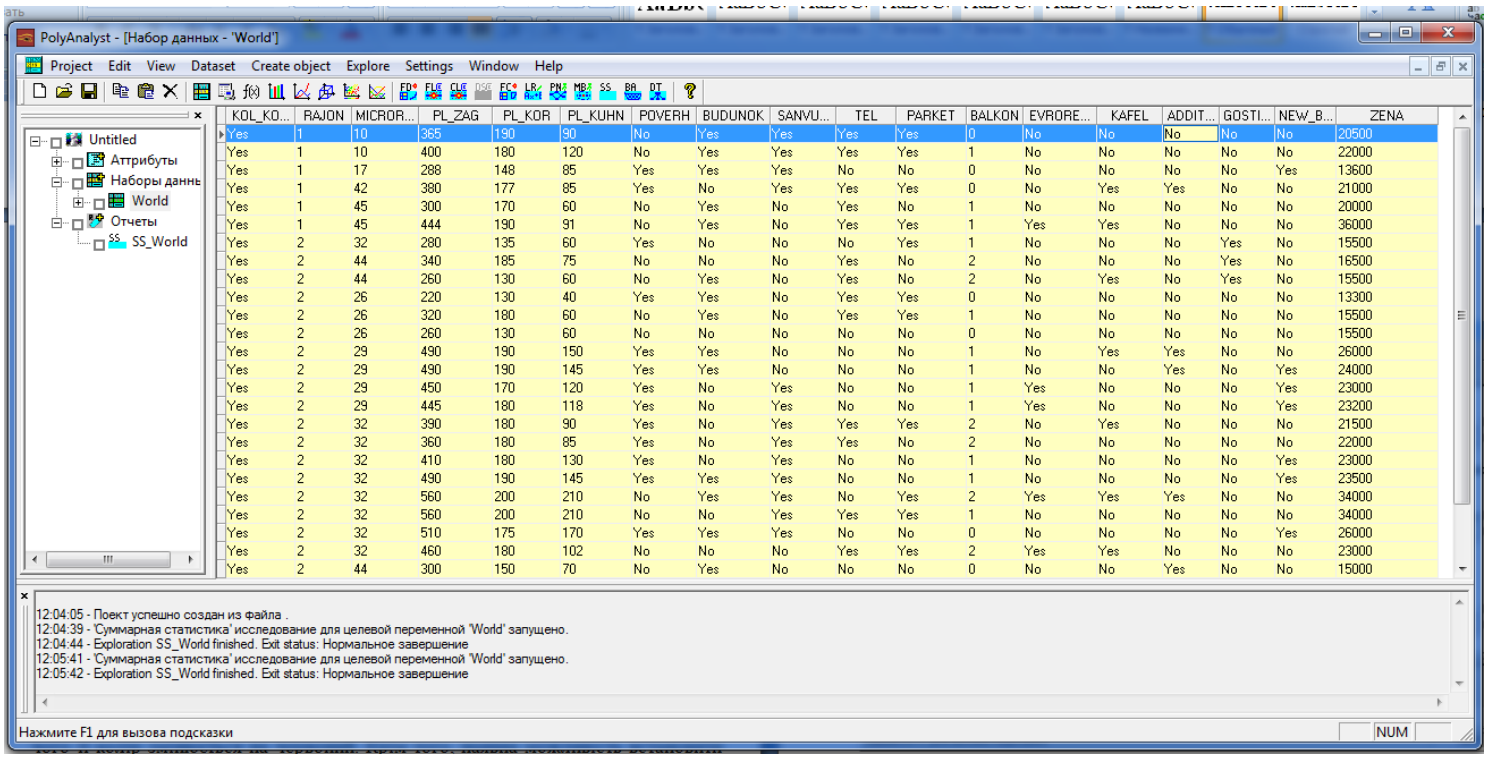


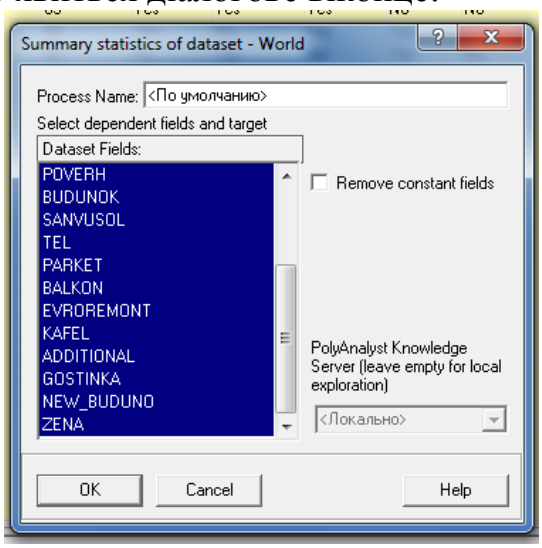
Рис. 9. Екран пакету PolyAnalyst після створення нового проекту

Два рази клікнувши на об’єкті **World** в піддиректорії **Наборы данных** можна відкрити та переглянути імпортовані дані.

4. PolyAnalyst надає можливості використовувати різні алгоритми та підходи для аналізу даних, вважається за необхідне (в т.ч. для цілей Курсового проектування) спочатку використати можливості модуля **SS** для отримання попередньої інформації по набору даних користувача.

Для початку аналізу, клікніть по найменуванню завантаженого набору даних у вікні проекту зліва, після цього стануть активними іконки доступних алгоритмів (в верхній частині вікна - в головному меню.

Клікніть по іконці SS, з’явиться діалогове віконце:



Клікніть на кнопці ok, з'явиться на екрані відповідний звіт роботи модуля SS.

Його результати треба роздрукувати та стисло проаналізувати та написати власні висновки в звіті по ЛР.

PolyAnalyst - [Отчет - 'SS_World']

Project Edit View Report Create object Explore Settings Window Help

- Untitled
  - Аттрибуты
  - Наборы данны
    - World
  - Отчеты
    - SS_World

Text

Последний результат был получен 27.01.2001, 12:26 для таблицы: World. 0 часов 0 минут прошло с момента старта. Процесс запущен на таблице 'World' с включёнными атрибутами: KOL_KOMNAT, RAJON, MICRORAJON, PL_ZAG, PL_KOR, PL_KUHN, POVERH, BUDUNOK, SANVUSOL, TEL, PARKET, BALKON, EVROREMONT, KAFEL, ADDITIONAL, GOSTINKA, NEW_BUDUNO, ZENA

Вычислительный процесс был запущен на локальном компьютере.

Таблица содержит 18 атрибутов и 28 записей.

Числовые и целочисленные атрибуты:	Значений	Среднее	Стд.откл.	Мин	Макс	Расстояние	Медиана	Мода	Разл.знач.
RAJON	28	1.786	0.4179	1	2	1	2		
MICRORAJON	28	32.68	9.862	10	45	35	32		
PL_ZAG	28	395.1	93.13	220	560	340	400		
PL_KOR	28	173.2	21.26	130	200	70	180		
PL_KUHN	28	106	44.41	40	210	170	91		
BALKON	28	0.9286	0.7164	0	2	2	1		
ZENA	28	2.181e+004	5869	1.33e+004	3.6e+004	2.27e+004	2.2e+004		

Логические (да/нет) атрибуты:	Значений	Кол-во 1	Кол-во 0
KOL_KOMNAT	28	28 (100%)	0 (0%)
POVERH	28	15 (53%)	13 (46%)
BUDUNOK	28	14 (50%)	14 (50%)
SANVUSOL	28	16 (57%)	12 (42%)
TEL	28	14 (50%)	14 (50%)
PARKET	28	12 (42%)	16 (57%)
EVROREMONT	28	5 (17%)	23 (82%)
KAFEL	28	8 (28%)	20 (71%)
ADDITIONAL	28	6 (21%)	22 (78%)
GOSTINKA	28	3 (10%)	25 (89%)
NEW_BUDUNO	28	9 (32%)	19 (67%)

12:04:39 - 'Суммарная статистика' исследование для целевой переменной 'World' запущено.
12:04:44 - Exploration SS_World finished. Exit status: Нормальное завершение
12:05:41 - 'Суммарная статистика' исследование для целевой переменной 'World' запущено.
12:05:42 - Exploration SS_World finished. Exit status: Нормальное завершение
12:26:00 - 'Суммарная статистика' исследование для целевой переменной 'World' запущено.
12:26:01 - Exploration SS_World finished. Exit status: Нормальное завершение

Нажмите F1 для вызова подсказки

NUM

SAMPLE

5. The Find Dependencies algorithm is used to find relationships and dependencies in the data quickly without finding the specific form of those dependencies. Rather, it tells you just how strong the connections between attributes are, so that you can include only the most influential attributes in the explorations performed by other machine learning engines. Also, the Find Dependencies algorithm produces a predictive rule in the form of table, which can be multi-dimensional – a format different from the presentation by Find Laws, PolyNet Predictor, Linear Regression, Nearest Neighbor, and other algorithms of PolyAnalyst. Multi-dimensional tables are represented on the screen as a set of two-dimensional tables corresponding to different slices of the complete table. In addition, Find Dependencies can be used to identify points that do not obey the dependencies found in most of the data, thus allowing identification of "stray" data points and errors in the data.

The Find Dependencies engine will work on almost any data, and can handle any of PolyAnalyst's data types so long as the target attribute and at least one other included attribute are numerical. Normally, in fact, you would use every attribute available to you in a Find Dependencies exploration. One of the main purposes of Find Dependencies is to find out which attributes are more important, so as to be able to work with less data in future explorations. Find Dependencies works quickly compared to the more sophisticated algorithms like Find Laws, PolyNet Predictor, and Nearest Neighbor.

Problems to Solve

The Strict version of Find Dependencies can be used to find out which attributes have the strongest influence on the target attribute. For instance, before running long Find Laws or PolyNet Predictor explorations on a list of customers, you may want to first run Find Dependencies on them to determine which attributes are truly influencing, and eliminate from the Find Laws exploration any that Find Dependencies ignored.

The Liberal version of Find Dependencies can be used to find outlying points in a dataset. It is very lenient on what can be considered a functional dependency – as a result, most points will fall into the dependent subpopulation. However, really unusual points – which could represent fraudulent transactions, data entry errors, or simply exceptional cases – will not. Thus, Find Dependencies can eliminate bad data from your exploration (unless the "bad data" is the actual target of your exploration, as in the case of fraudulent transactions, in which Find Dependencies' Liberal algorithm may be the central focus of your exploration.)

Find Dependencies can be used to eliminate the effect of stray data points from your exploration. Many PolyAnalyst exploration methods are vulnerable to the problem that a few outlying points with extreme values can considerably throw off the resulting expression. The Liberal version of Find Dependencies can be used to create a dataset – called the dependent subpopulation – which excludes these points.

The target attribute

The target attribute of a Find Dependencies exploration must be of a numerical data type. It works with continuous functions, and requires a data type that can represent continuous variation. Find Dependencies cannot target a category, string, or yes/no result. For these, you must use Classify or Nearest Neighbor. Find Dependencies is often useful for solution of classification problems - but in this case yes/no-type target variable should be converted to numerical by the rule if (YesNoVar, 1,0).

When to Use This Algorithm

The strict version of Find Dependencies is often used as one of the first steps of data mining. Once you have a target attribute in mind (either because it was the purpose of your

investigation in the first place, or due to hints obtained through the use of undirected methods like Clustering), you can begin investigating with Find Dependencies. Many of PolyAnalyst's exploration engines, especially PolyNet Predictor, take significantly more time for each additional attribute explored. By running Find Dependencies on all attributes first, you can find out which attributes are most important to include. If Find Dependencies returns only a small number of attributes, and you feel you need more in the exploration, you can run Find Dependencies again, this time excluding the attributes found the first time. Without these strong dependencies shown, Find Dependencies will find weaker links in the data that may still be important. Once you have enough attributes through the iterative application of Find Dependencies, you can run Find Laws, PolyNet Predictor, or Classify including only those attributes that Find Dependencies found to be important.

Note that in real data, often all the attributes are connected in some ways, and if you run Find Dependencies enough times (each time excluding the attributes found important in the previous exploration), you will sometimes eventually find every attribute to be important to the target. It is up to you to determine how many attributes you need to include in Find Laws or other exploration engines – this is a trade-off between the potential accuracy of the prediction and the amount of time it will take. However, Find Dependencies helps you with this decision by providing measures of statistical significance in its report.

The Liberal version of Find Dependencies can be used at the very beginning of data mining, before any other engines are used, to eliminate "stray" data points that may be the result of errors in the data. In addition, it can be used later in the data mining process to find unusual points – if the data consisted of bank transactions, these "stray" points may be exactly what you are looking for.

Also the results obtained by Find Dependencies can be used a final model when the explored data include many categorical attributes. In this case the representation of a prediction model in a tabular form might be most adequate.

Another interesting feature of Find Dependencies is its ability to recognize so called partial dependencies. For example, z may depend on x - but only when $y>0$. Find Dependencies will notice this fact, producing a prediction table in which the cells corresponding to positive values of y will have small values in their cell's standard error, while the cells with $y<0$ will be characterized by standard errors close to 100%.

The Output

The Find Dependencies report's first line is often the most important – it lists the most influencing parameters. These are the attributes found to be most related to the target attribute, and thus most useful in its prediction. Next, is the number of points obeying the dependence – this shows how many of the points in the dataset follow the dependence discovered. If this number is very high compared to the total number of points in the dataset, this may indicate that the few points not found in the dependence may be of interest as exceptions to the rule, or may be erroneous data.

Following these data are the error rates for the exploration. The standard deviation of the target attribute in the dataset as a whole and in the subpopulation (the points obeying the dependence) is displayed, along with the interval of errors for 70%, 90%, and 97% of the data points. These statistics show how strong the dependence is, and are explained fully in Appendix D.

The next output of Find Dependencies is the dependencies table. The table divides the data into segments along two axes – the two axes being the attributes that had the strongest influence on the target attribute by default. Each cell is color-coded depending on whether or not it is part of the dependent subpopulation – cells that do not fall into the subpopulation are white. Within each cell, five numbers are listed – the predicted value, the number of points in the cell, the standard error of those points, the number of points in the cell that fall into the dependent subpopulation, and the standard error of those points.

The key numbers in each cell are usually the number of points in the cell and the number of points in the subpopulation. This shows you how many points within each range obey the found dependence compared to how many total points there are. This indicates which ranges show a stronger dependence. The standard error statistics are measures of significance; they are discussed in detail in Appendix D.

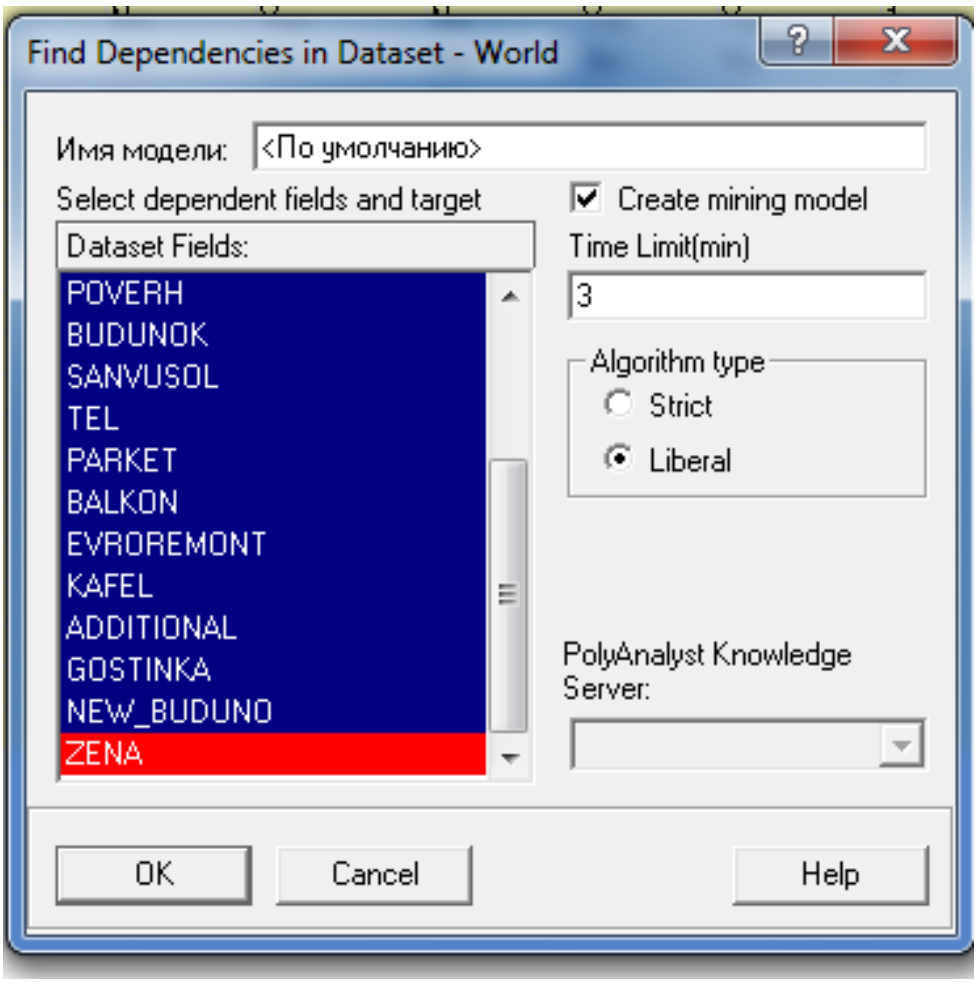
In some cases, more than two attributes may have been used to form the cells. Since a table in 3 or more dimensions can't be displayed, you can choose which two attributes are used to form the axes by selecting them from the "Row Heading" and "Column Heading" drop-down boxes. Since this two-dimensional table is only a slice of the full *n*-dimensional table, you must also select ranges for all the attributes not displayed – for this purpose, there is a drop-down box listed for each attribute. Select the appropriate ranges, and the proper section of the table will be displayed. There is no effect to selecting ranges for the attributes that *are* displayed – the full range is shown for these attributes.

Finally, the Find Dependencies report contains three standard graphs – Predicted vs. Real, Predicted and Real vs. Record Number, and Residuals. See the previous chapter for information on reading PolyAnalyst standard graphs. In addition, Find Dependencies outputs a new Rule to the Rules collection. This Rule cannot be displayed or edited (it actually consists of the dependencies table), but can be applied to other datasets to determine if records fall into the dependent subpopulation.

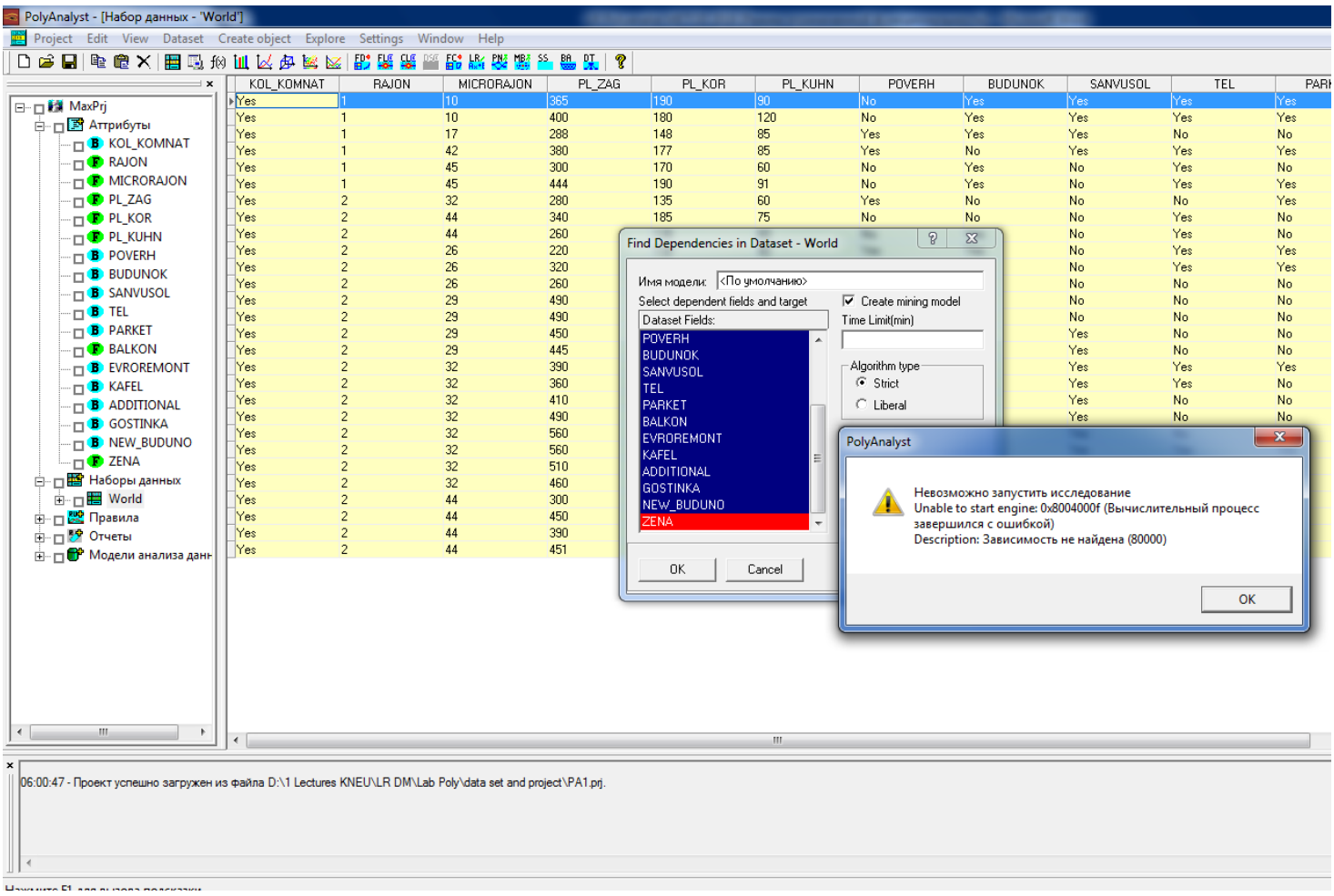
The Technology

Find Dependencies analyzes distribution patterns of points in different regions of a complete multidimensional space of all attribute values. The results of Find Dependencies are much more useful than that of a simple correlation analysis, since it will find relationships involving multiple attributes together influencing the target attribute, rather than just 1-to-1 correlations. In addition, the Find Dependencies algorithm will correctly find nonlinear dependencies.

For further technical details about the technology behind Find Dependencies, see Appendix C. For more information about measures of accuracy and significance with Find Dependencies, see Appendix D.



When you select **Explore | Find Dependencies**, you will be given the option to select/deselect attributes to consider (by right clicking on them) and a target attribute (by double-clicking). In addition, you can fill in a termination condition by entering a value into "Time Limit." Find Dependencies will finish whenever the time runs out or no statistically significant improvement is expected. In addition, you can choose either the Strict or the Liberal version of the algorithm – which is chosen depends on the type of problem you wish to solve.



6. PolyAnalyst пропонує цілий ряд потужних алгоритмів ІАД для пошуку закономірностей між змінними. Однак найбільш універсальним та потужним, з огляду на задачі дейтамайнінгу, є модуль Find Laws (пошук законів), він використовує алгоритм **SKAT** (Symbolic Knowledge Acquisition Technology).

Алгоритм призначений для автоматичного знаходження в даних нелінійних залежностей та подання результатів у вигляді математичних формул, які можуть містити блоки умов. Здатність модуля FL автоматично будувати велику різноманітність математичних конструкцій робить його унікальним інструментом пошуку знань у символьному вигляді.

Пошук законів ґрунтується на технології отримання символічних знань (SKAT), алгоритмі, розробленому Megaputer Intelligence. Використовуючи методи еволюційного програмування, він розробляє раціональні вирази високого ступеня та інші математичні конструкції, які можуть виражати собою широкий спектр явищ і набагато краще відображають нелінійні залежності, ніж лінійні регресії і аналогічні методи. Крім того, на відміну від нейронних мереж, SKAT видає зрозумілий символічний вираз (полінном), щоб людина могла читати і розуміти, як працює модель. Еволюційний алгоритм використовує ідею так званих узагальнюючих

перетворень, щоб засновувати нові вирази на базі раніше розроблених, зберігаючи певні позитивні риси, і розвиваючись в залежності від того, який вираз найкраще апроксимує. Таким чином, пошук законів працює без знання виду закономірності. Пошук законів також включає ретельне тестування значущості з використанням рандомізованого методу тестування. В результаті такого тестування результатів, зберігаються тільки закономірності з реальною прогностичної силою.

Алгоритм заснований на технології еволюційного програмування та ефективно працює з атрибутами типу Float.

Алгоритм **FL** працює ефективно, якщо невелика частина атрибутів є категоріальною, або, якщо категоріальні атрибути мають два значення, наприклад 0 та 1. Якщо ж у вас основна частина атрибутів є категоріальними з багатьма значеннями, та/або атрибути мають тип даних string – треба використовувати для передбачення алгоритми класифікації або CBR.

Оптимальні розміри набору даних для цього алгоритму:

- мінімум 30 записів;
- оптимально – біля 20 000 записів;
- максимум – до 1 000 000 записів.

Для запуску модуля **FL** необхідно клікнути правою кнопкою миші на сформованому наборі даних **World**, та у контекстному вікні обрати **Explore → Find Laws.** Після чого відкриється вікно діалогу модуля **FL** (рис. 10).

В цьому вікні необхідно на цільовій змінній клікнути два рази мишою, після чого її колір змінюється на червоний.

В реальних задачах ІАД, важливиим є можливості встановити ліміт часу на процес пошуку та допустиму стандартну помилку.

Для початку процесу дейтамайнінгу натискаємо кнопку **ОК**.

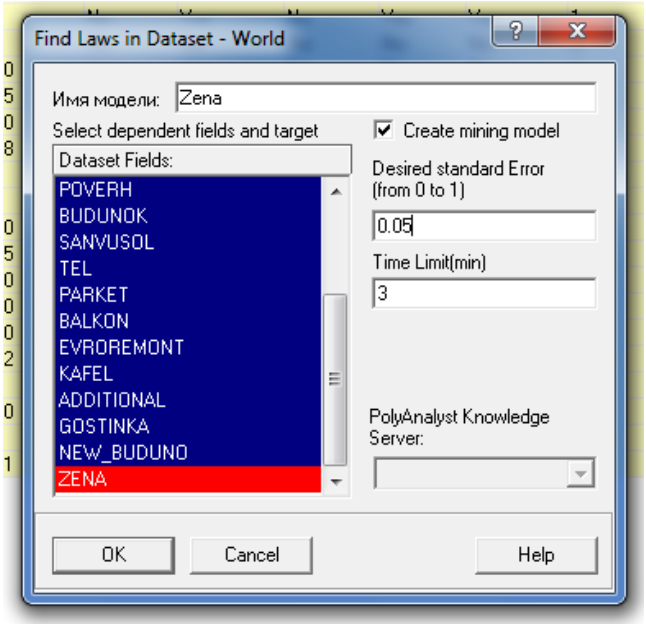
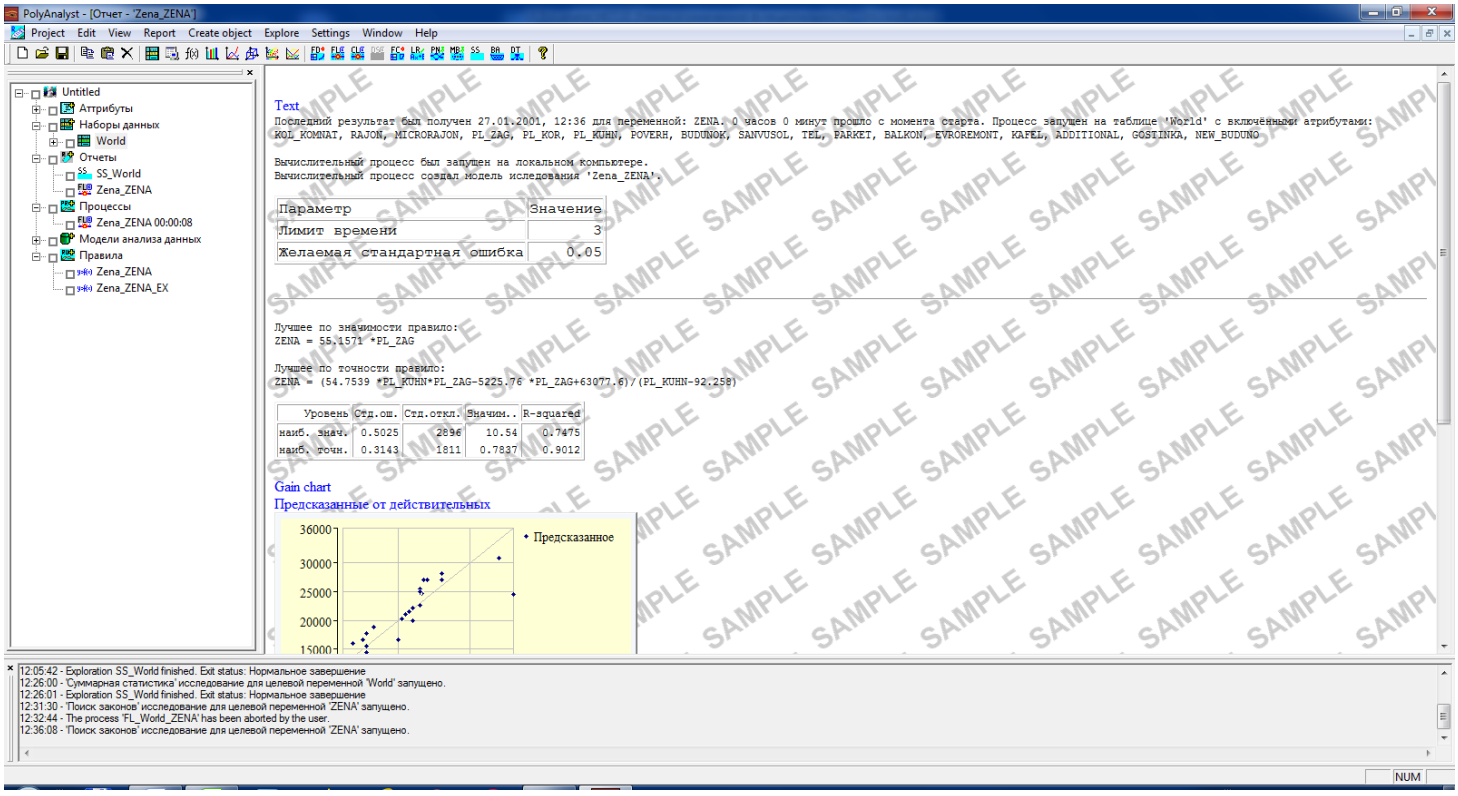


Рис. 10. Діалогове вікно модуля FL

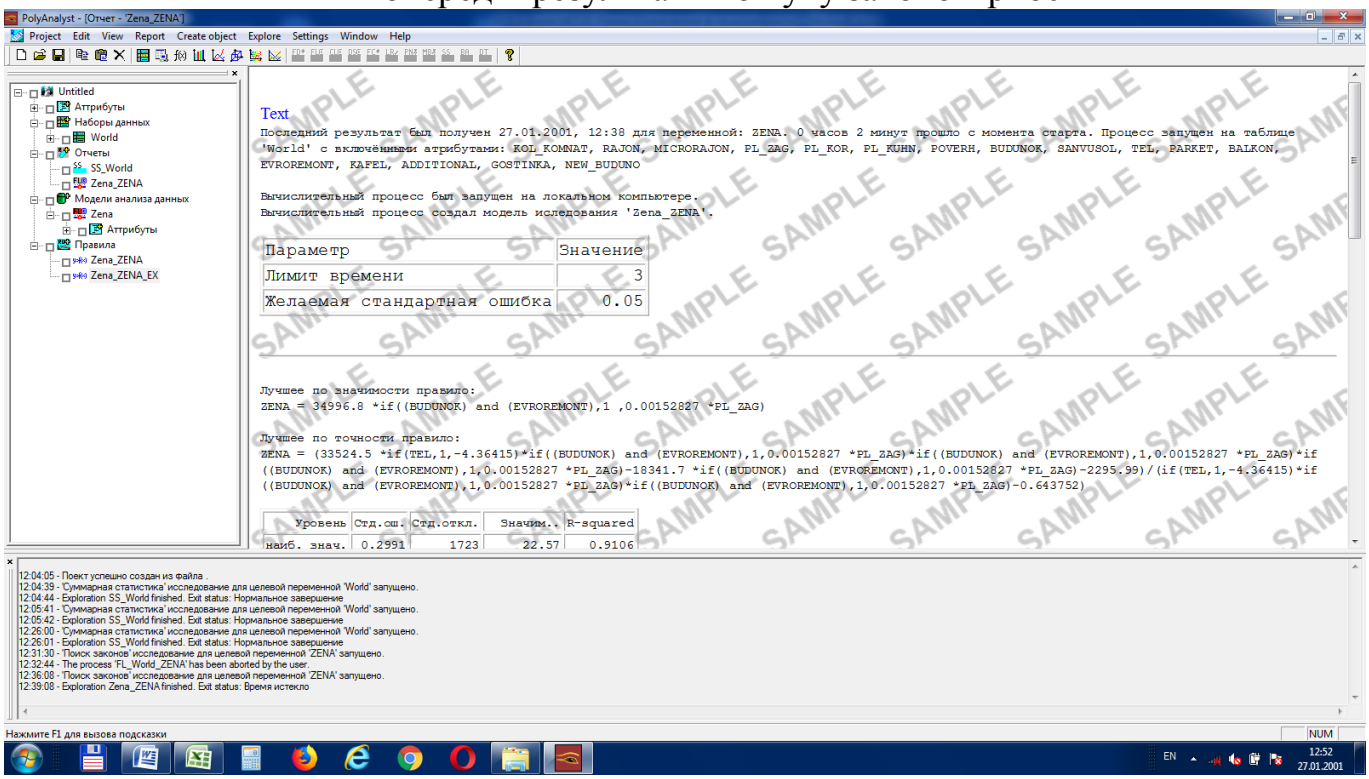
7. Після початку роботи алгоритму, зверніть увагу на трансформацію та уточнення шуканої закономірності з плином часу.

Після закінчення обчислень у робочій області проекта з’явиться дві піддиректорії: **Правила** та **Отчеты** (рис. 11).

Порівняйте результати попередні та фінальні.



Попередні результати пошуку закономірності



Фінальні результати пошуку закономірності, обмежені лімітом часу

Рис. 11. Загальне вікно Polyanalyst після процесу дейтамайнінгу

Зверніть вашу увагу на вікно ходу виконання проекту, у нижній частині вікна Polyanalyst.

8. Як видно на рис. 11, сформовано два правила під назвами: **FL_y** (найкраще по значущості правило) та **FL_y_Ex** (найкраще по точності правило). Префіх **FL** означає, що правило сформовано за допомогою модуля **Find Laws**. Далі у назві правила стоїть залежна змінна (в нашому прикладі - **y**).

Два рази клацнувши по зображенню правила відкриємо для перегляду вікно редагування правила (рис. 12.).

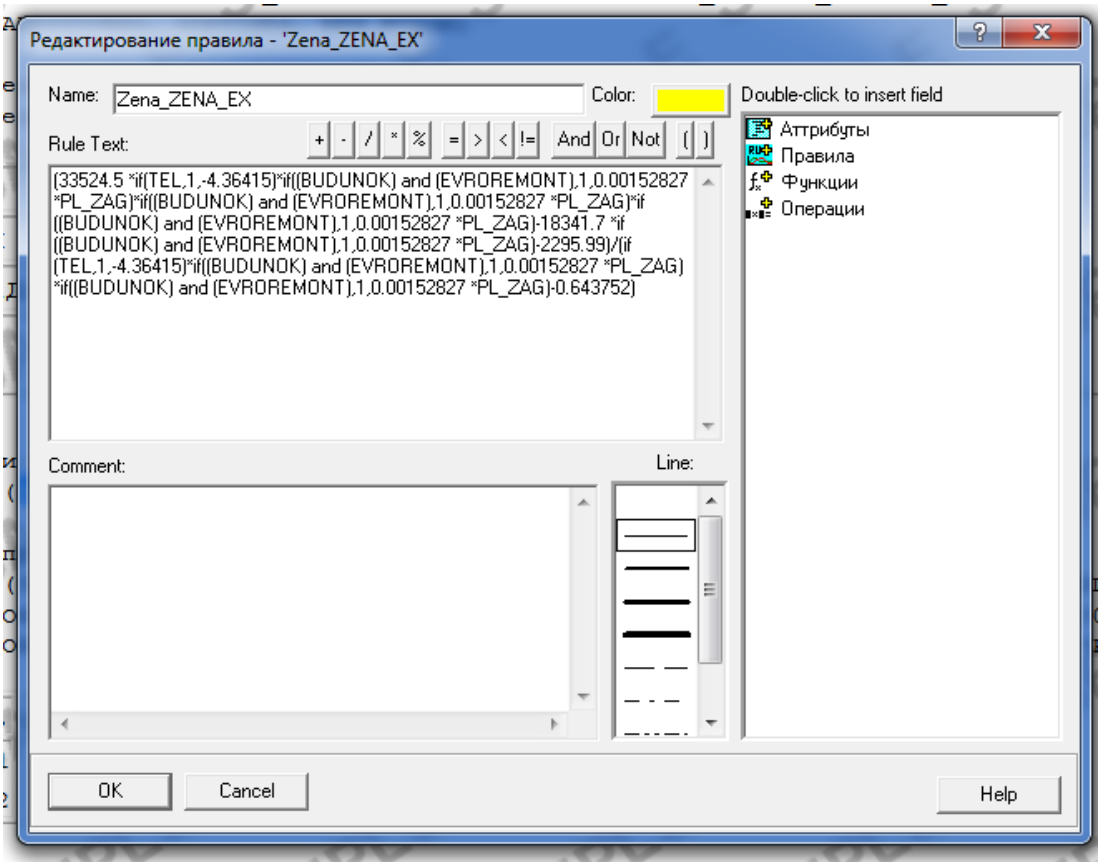


Рис. 12. Діалогове вікно редагування правил

9. В основному вікні PolyAnalyst автоматично відобразиться сформований звіт, який буде складатися з 2-х частин: текстової та графічної. Також його можна переглянути два рази клікнувши по піктограммі звіту у робочій області проекту.

Текстова частина зввыта - подана нижче:

Text
Последний результат был получен 27.01.2001, 12:38 для переменной: ZENA. 0 часов 2 минут прошло с момента старта. Процесс запущен на таблице 'World' с включёнными атрибутами: KOL_KOMNAT, RAJON, MICRORAJON, PL_ZAG, PL_KOR, PL_KUHN, POVERH, BUDUNOK, SANVUSOL, TEL, PARKET, BALKON, EVROREMONT, KAFEL, ADDITIONAL, GOSTINKA, NEW_BUDUNO

Вычислительный процесс был запущен на локальном компьютере.
Вычислительный процесс создал модель иследования 'Zena_ZENA'.

Параметр	Значение
Лимит времени	3
Желаемая стандартная ошибка	0.05

Лучшее по значимости правило:
ZENA = 34996.8 *if((BUDUNOK) and (EVROREMONT),1 ,0.00152827 *PL_ZAG)

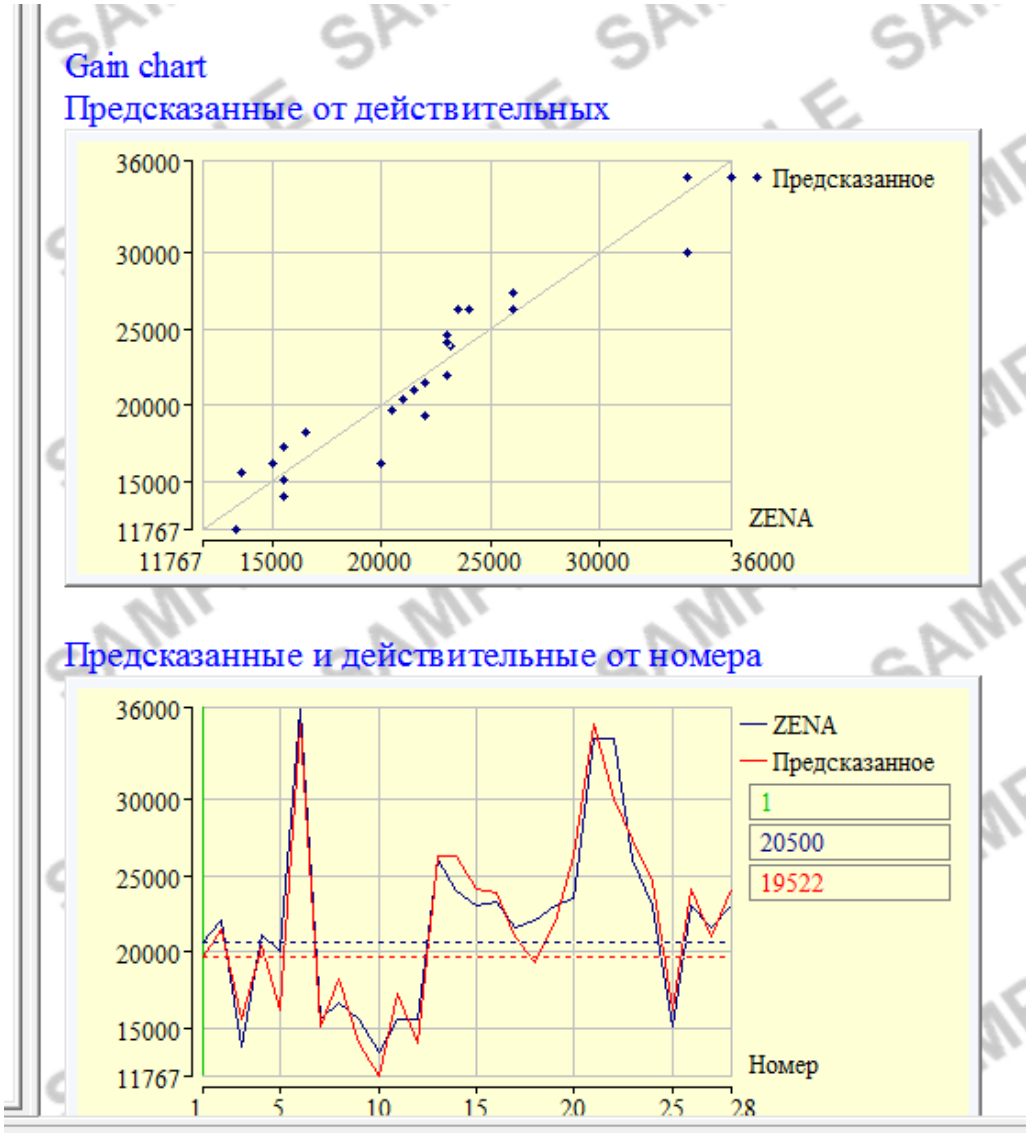
Лучшее по точности правило:
ZENA = (33524.5 *if(TEL,1,-4.36415)*if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)*if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)*if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)-18341.7 *if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)-2295.99)/(if(TEL,1,-4.36415)*if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)*if((BUDUNOK) and (EVROREMONT),1,0.00152827 *PL_ZAG)-0.643752)

Уровень	Стд.ош.	Стд.откл.	Значим..	R-squared
наиб. знач.	0.2991	1723	22.57	0.9106
наиб. точн.	0.2102	1211	не значим.	0.9558

Як бачимо, в текстовій частині наведено:
- технічна інформація щодо процесу дейтамайнінгу;
- безпосередньо знайдені правила: “лучшее по значимости правило” и “лучшее по точности правило”. В нашому прикладі ці правила однакові, однак в більшості реальних ситуацій цього не відбудеться. На практиці, більш просте та надійне “найкраще по значущості правило” використовується у більшості випадків. А “найкраще по точності правило”, використовується у виключних випадках, коли найменше покращення в точності необхідно.
- статистичні показники, що характеризують різницю між вхідними даними та обчисленими за допомогою знайдених правил. А саме: стандартну похибку; стандартне відхилення; індекс значущості (якщо більше 3, то правило невипадкове); коефіцієнт R-squared (чим вище, тим більш точне правило).

10. Графічна частина звіту містить 4 сформовані графіки:
- Gain Chart (в даному прикладі не використовується, оскільки застосовується в маркетинговому аналізі,);
- Predicted Versus Real (відображає передбачені за допомогою знайденого правила значення в порівнянні із вхідними даними.Отже чим ближче графік до діагоналі, тим краща якість рішення);
- Predicted and Real Versus Counter (відображає передбачені за допомогою знайденого правила значення, вхідні дані та порядковий номер вхідного набору даних);

- Residuals (відображає розподіл девіації).



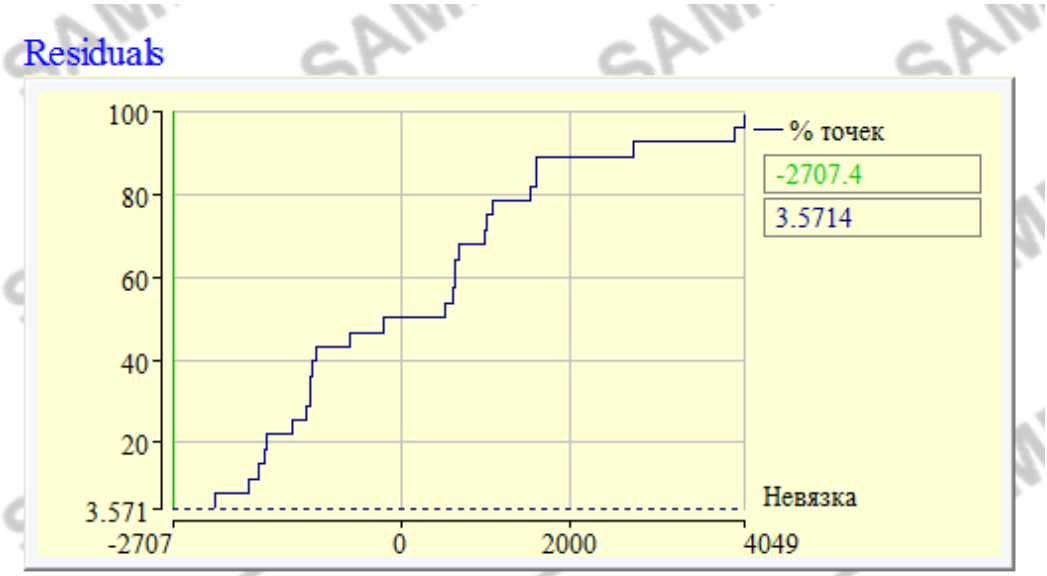


Рис. 13. Графіки зі звіту, що характеризують якість знайденої закономірності

В звіті по ЛР, необхідно, крім роздруківки даних графіків, обов'язково (з використанням даних графіків) дати власні висновки щодо якості знайденої закономірності.

11. В звіті по даній лабораторній роботі обов'язково повинно міститися:

- роздруківка оригінального набору даних або його фрагменту з мета даними (достатньо 1 стор);
- роздруківка фрагменту вікна з завантаженим в систему PolyAnalyst набором вхідних даних;
- роздруківка згенерованих системою звіту та графіків модуля **SS** зі стислими висновками;
- роздруківка згенерованих системою звіту та графіків модуля **FD** зі стислими висновками щодо набору атрибутів для наступного аналізу;
- роздруківка згенерованих системою звіту та графіків модуля **FL** з власними висновками щодо самої закономірності;
- з використанням графіків, що характеризують якість знайденої закономірності, необхідно навести власні висновки щодо якості знайденої закономірності.

Список ресурсів, що містять ON-LINE банки даних для завантаження в пакет Polyanalyst та подальшого проведення інтелектуального аналізу даних в ЛР:

UCI KDD Archive: Data Sets
Data Sets from SGI
UCal Irvine Machine Learning Repository
Criminal Justice Data a NACJD
Social Science Data
Data Sets
IDS data sets
Data Sets for Data Mining
Competition Data Set
UCI Machine learning repository
Quest data repository
KDNuggets

І інші з https://www.kdnuggets.com/datasets/index.html

5.
Find Dependencies

The Find Dependencies algorithm is used to find relationships and dependencies in the data quickly without finding the specific form of those dependencies. Rather, it tells you just how strong the connections between attributes are, so that you can include only the most influential attributes in the explorations performed by other machine learning engines. Also, the Find Dependencies algorithm produces a predictive rule in the form of table, which can be multi-dimensional – a format different from the presentation by Find Laws, PolyNet Predictor, Linear Regression, Nearest Neighbor, and other algorithms of PolyAnalyst. Multi-dimensional tables are represented on the screen as a set of two-dimensional tables corresponding to different slices of the complete table. In addition, Find Dependencies can be used to identify points that do not obey the dependencies found in most of the data, thus allowing identification of "stray" data points and errors in the data.

The Find Dependencies engine will work on almost any data, and can handle any of PolyAnalyst's data types so long as the target attribute and at least one other included attribute are numerical. Normally, in fact, you would use every attribute available to you in a Find Dependencies exploration. One of the main purposes of Find Dependencies is to find out which attributes are more important, so as to be able to work with less data in future explorations. Find Dependencies works quickly compared to the more sophisticated algorithms like Find Laws, PolyNet Predictor, and Nearest Neighbor.

When you select **Explore | Find Dependencies**, you will be given the option to select/deselect attributes to consider (by right clicking on them) and a target attribute (by double-clicking). In addition, you can fill in a termination condition by entering a value into "Time Limit." Find Dependencies will finish whenever the time runs out or no statistically significant improvement is expected. In addition, you can choose either the Strict or the Liberal version of the algorithm – which is chosen depends on the type of problem you wish to solve.

The Strict version of Find Dependencies can be used to find out which attributes have the strongest influence on the target attribute. For instance, before running long Find Laws or PolyNet Predictor explorations on a list of customers, you may want to first run Find Dependencies on them to determine which attributes are truly influencing, and eliminate from the Find Laws exploration any that Find Dependencies ignored.

The Liberal version of Find Dependencies can be used to find outlying points in a dataset. It is very lenient on what can be considered a functional dependency – as a result, most points will fall into the dependent subpopulation. However, really unusual points – which could represent fraudulent transactions, data entry errors, or simply exceptional cases – will not. Thus, Find Dependencies can eliminate bad data from your exploration (unless the "bad data" is the actual target of your exploration, as in the case of fraudulent transactions, in which Find Dependencies' Liberal algorithm may be the central focus of your exploration.)

Find Dependencies can be used to eliminate the effect of stray data points from your exploration. Many PolyAnalyst exploration methods are vulnerable to the problem that a few outlying points with extreme values can considerably throw off the resulting expression. The Liberal version of Find Dependencies can be used to create a dataset – called the dependent subpopulation – which excludes these points.

The target attribute of a Find Dependencies exploration must be of a numerical data type. It works with continuous functions, and requires a data type that can represent continuous variation. Find

Dependencies cannot target a category, string, or yes/no result. For these, you must use Classify or Nearest Neighbor. Find Dependencies is often useful for solution of classification problems - but in this case yes/no-type target variable should be converted to numerical by the rule if (YesNoVar, 1,0).

Найти зависимости

Алгоритм Find Dependencies используется для быстрого поиска зависимостей и зависимостей в данных, не обнаруживая конкретную форму этих зависимостей. Скорее, это говорит вам, насколько сильны связи между атрибутами, поэтому вы можете включать только самые влиятельные атрибуты в исследованиях, выполненных другими машинами для машинного обучения. Кроме того, алгоритм Find Dependencies создает правило прогнозирования в виде таблицы, которое может быть многомерным - формат, отличный от представления Find Laws, PolyNet Predictor, Linear Regression, Nearest Neighbor и других алгоритмов PolyAnalyst. Многомерные таблицы представлены на экране как набор двумерных таблиц, соответствующих различным фрагментам полной таблицы. Кроме того, Find Dependencies могут использоваться для определения точек, которые не подчиняются зависимостям, обнаруженным в большинстве данных, что позволяет идентифицировать «блуждающие» точки данных и ошибки в данных.

Механизм Find Dependencies будет работать практически с любыми данными и может обрабатывать любые типы данных PolyAnalyst, если атрибут target и, по крайней мере, один дополнительный атрибут являются численными. Обычно, фактически, вы должны использовать каждый атрибут, доступный вам в поиске Find Dependencies. Одна из основных целей Find Dependencies - выяснить, какие атрибуты важнее, чтобы иметь возможность работать с меньшим количеством данных в будущих исследованиях. Поиск зависимостей работает быстро по сравнению с более сложными алгоритмами, такими как Find Laws, PolyNet Predictor и Nearest Neighbor.

Когда вы выберите «Исследуйте | Найти зависимости, вам будет предложено выбрать / отменить выбор атрибутов (щелкнув правой кнопкой мыши их) и целевого атрибута (двойным щелчком). Кроме того, вы можете заполнить условие завершения, введя значение в «Ограничение времени». «Найти зависимости завершатся всякий раз, когда заканчивается время или не ожидается никакого статистически значимого улучшения. Кроме того, вы можете выбрать либо строковую, либо либеральную версию алгоритма, который выбран, зависит от типа проблемы, которую вы хотите решить.

Строгая версия Find Dependencies может использоваться для определения того, какие атрибуты оказывают самое сильное влияние на целевой атрибут. Например, прежде чем запускать поисковые запросы Find Laws или PolyNet Predictor в списке клиентов, вы можете сначала запустить Find Dependencies на них, чтобы определить, какие атрибуты действительно влияют, и исключить из исследования поиска законы, которые игнорируют Find Dependencies.

Либеральную версию Find Dependencies можно использовать для поиска точек в наборе данных. Это очень снисходительно к тому, что можно считать функциональной зависимостью - в результате большинство точек попадают в зависимую субпопуляцию. Однако действительно необычные моменты, которые могут представлять собой мошеннические транзакции, ошибки ввода данных или просто исключительные случаи - не будут. Таким образом, Find Dependencies могут устранить плохие данные из вашего исследования (если только «плохие данные» не являются фактической целью вашего исследования, как в случае мошеннических транзакций, в которых алгоритм поиска зависимостей «Либерал» может быть центром вашего исследования.)

Найти зависимости можно использовать для устранения влияния блуждающих точек данных из вашего исследования. Многие методы исследования PolyAnalyst уязвимы для проблемы, когда несколько точек с экстремальными значениями могут значительно сбросить полученное выражение. Либеральная версия Find Dependencies может использоваться для создания набора данных, называемого зависимой субпопуляцией, которая исключает эти точки.

Атрибут target исследования поиска зависит от типа данных. Он работает с непрерывными функциями и требует типа данных, который может представлять непрерывную вариацию. Найти зависимости не могут быть нацелены на категорию, строку или да / нет. Для этого вы должны использовать Classify или Nearest Neighbor. Найти зависимости часто полезно для решения задач классификации, но в этом случае целевая переменная yes / no-type должна быть преобразована в числовое значение по правилу if (YesNoVar, 1,0).

The strict version of Find Dependencies is often used as one of the first steps of data mining. Once you have a target attribute in mind (either because it was the purpose of your investigation in the first place, or due to hints obtained through the use of undirected methods like Clustering), you can begin investigating with Find Dependencies. Many of PolyAnalyst's exploration engines, especially PolyNet Predictor, take significantly more time for each additional attribute explored. By running Find Dependencies on all attributes first, you can find out which attributes are most important to include. If Find Dependencies returns only a small number of attributes, and you feel you need more in the exploration, you can run Find Dependencies again, this time excluding the attributes found the first time. Without these strong dependencies shown, Find Dependencies will find weaker links in the data that may still be important. Once you have enough attributes through the iterative application of Find Dependencies, you can run Find Laws, PolyNet Predictor, or Classify including only those attributes that Find Dependencies found to be important.

Note that in real data, often all the attributes are connected in some ways, and if you run Find Dependencies enough times (each time excluding the attributes found important in the previous exploration), you will sometimes eventually find every attribute to be important to the target. It is up to you to determine how many attributes you need to include in Find Laws or other exploration engines – this is a trade-off between the potential accuracy of the prediction and the amount of time it will take. However, Find Dependencies helps you with this decision by providing measures of statistical significance in its report.

The Liberal version of Find Dependencies can be used at the very beginning of data mining, before any other engines are used, to eliminate "stray" data points that may be the result of errors in the data. In addition, it can be used later in the data mining process to find unusual points – if the data consisted of bank transactions, these "stray" points may be exactly what you are looking for.

Also the results obtained by Find Dependencies can be used a final model when the explored data include many categorical attributes. In this case the representation of a prediction model in a tabular form might be most adequate.

Another interesting feature of Find Dependencies is its ability to recognize so called partial dependencies. For example, z may depend on x - but only when $y>0$. Find Dependencies will notice this fact, producing a prediction table in which the cells corresponding to positive values of y will have small values in their cell's standard error, while the cells with $y<0$ will be characterized by standard errors close to 100%.

Строгая версия Find Dependencies часто используется как один из первых шагов интеллектуального анализа данных. Если у вас есть целевой атрибут (потому что он был целью вашего расследования в первую очередь или из-за подсказок, полученных с помощью неориентированных методов, таких как Clustering), вы можете начать исследование с помощью Find Dependencies. Многие из поисковых систем PolyAnalyst, особенно PolyNet Predictor, занимают значительно больше времени для изучения каждого дополнительного атрибута. Сначала запустив Find Dependencies для всех атрибутов, вы можете узнать, какие атрибуты наиболее важны для включения. Если Find Dependencies возвращает только небольшое количество атрибутов, и вы чувствуете, что вам нужно больше в исследовании, вы можете снова запустить Find Dependencies, на этот раз, исключая атрибуты, найденные в первый раз. Без этих сильных зависимостей, Find Dependencies найдут более слабые ссылки в данных, которые все еще могут быть важными. Когда у вас будет достаточно атрибутов с помощью итеративного приложения Find Dependencies, вы можете запустить Find Laws, PolyNet Predictor или Classify, включая только те атрибуты, которые Find Dependencies считаются важными.

Обратите внимание, что в реальных данных часто все атрибуты связаны в некотором роде, и если вы запустите Find Dependencies достаточно раз (каждый раз, исключая атрибуты, которые были важны в предыдущем исследовании), вы иногда, в конечном итоге, обнаружите, что каждый атрибут будет важен для цель. Это зависит от вас, чтобы определить, сколько атрибутов вам нужно включить в Find Laws или другие поисковые системы - это компромисс между потенциальной точностью прогнозирования и количеством времени, которое потребуется. Однако Find Dependencies помогают вам в этом решении, предоставляя статистические данные в своем отчете.

Либеральная версия Find Dependencies может использоваться в самом начале процесса интеллектуального анализа данных до того, как будут использоваться другие двигатели, чтобы исключить «блуждающие» точки данных, которые

могут быть результатом ошибок в данных. Кроме того, он может быть использован позже в процессе интеллектуального анализа данных, чтобы найти необычные точки - если данные состоят из банковских транзакций, эти «бродячие» точки могут быть именно тем, что вы ищете.
Кроме того, результаты, полученные с помощью Find Dependencies, могут использоваться для окончательной модели, когда исследуемые данные включают в себя множество категориальных атрибутов. В этом случае представление модели прогнозирования в табличной форме может быть наиболее адекватным.
Еще одна интересная особенность Find Dependencies - это способность распознавать так называемые частичные зависимости. Например, z может зависеть от x - но только при y> 0. Find Dependencies заметили бы этот факт, создав таблицу прогнозирования, в которой ячейки, соответствующие положительным значениям y, будут иметь небольшие значения в стандартной ошибке их ячейки, а ячейки с y <0 будут характеризоваться стандартными ошибками, близкими к 100%.

Find Dependencies analyzes distribution patterns of points in different regions of a complete multidimensional space of all attribute values. The results of Find Dependencies are much more useful than that of a simple correlation analysis, since it will find relationships involving multiple attributes together influencing the target attribute, rather than just 1-to-1 correlations. In addition, the Find Dependencies algorithm will correctly find nonlinear dependencies.

Following these data are the error rates for the exploration. The standard deviation of the target attribute in the dataset as a whole and in the subpopulation (the points obeying the dependence) is displayed, along with the interval of errors for 70%, 90%, and 97% of the data points. These statistics show how strong the dependence is, and are explained fully in Appendix D.

The next output of Find Dependencies is the dependencies table. The table divides the data into segments along two axes – the two axes being the attributes that had the strongest influence on the target attribute by default. Each cell is color-coded depending on whether or not it is part of the dependent subpopulation – cells that do not fall into the subpopulation are white. Within each cell, five numbers are listed – the predicted value, the number of points in the cell, the standard error of those points, the number of points in the cell that fall into the dependent subpopulation, and the standard error of those points.

The key numbers in each cell are usually the number of points in the cell and the number of points in the subpopulation. This shows you how many points within each range obey the found dependence compared to how many total points there are. This indicates which ranges show a stronger dependence. The standard error statistics are measures of significance; they are discussed in detail in Appendix D.

In some cases, more than two attributes may have been used to form the cells. Since a table in 3 or more dimensions can't be displayed, you can choose which two attributes are used to form the axes by selecting them from the "Row Heading" and "Column Heading" drop-down boxes. Since this two-dimensional table is only a slice of the full *n*-dimensional table, you must also select ranges for all the attributes not displayed – for this purpose, there is a drop-down box listed for each attribute. Select the appropriate ranges, and the proper section of the table will be displayed. There is no effect to selecting ranges for the attributes that *are* displayed – the full range is shown for these attributes.

In addition, Find Dependencies outputs a new Rule to the Rules collection. This Rule cannot be displayed or edited (it actually consists of the dependencies table), but can be applied to other datasets to determine if records fall into the dependent subpopulation.

Найти зависимости анализирует шаблоны распределения точек в разных областях полного многомерного пространства всех значений атрибутов. Результаты Find Dependencies гораздо более полезны, чем результаты простого корреляционного анализа, поскольку он будет определять отношения, включающие в себя несколько атрибутов, влияющих на атрибут target, а не только на 1-к-1 корреляции. Кроме того, алгоритм Find Dependencies корректно найдет нелинейные зависимости.
После этих данных приведены коэффициенты ошибок для исследования. Отображается стандартное отклонение целевого атрибута в наборе данных в целом и в субпопуляции (точки, подчиняющиеся зависимости), а также интервал ошибок для 70%, 90% и 97% точек данных. Эти статистические данные показывают, насколько сильна зависимость, и полностью объясняются в Приложении D.

Следующий вывод Find Dependencies - это таблица зависимостей. Таблица делит данные на сегменты вдоль двух осей - две оси являются атрибутами, которые по умолчанию влияли на атрибут назначения. Каждая ячейка кодируется цветом в зависимости от того, является ли она частью зависимой субпопуляции - ячейки, которые не попадают в субпопуляцию, являются белыми. В каждой ячейке перечислены пять чисел - предсказанное значение, количество точек в ячейке, стандартная ошибка этих точек, количество точек в ячейке, которые попадают в зависимую субпопуляцию, и стандартную ошибку этих точек.
Номера клавиш в каждой ячейке обычно представляют собой количество точек в ячейке и количество точек в субпопуляции. Это показывает вам, сколько точек в каждом диапазоне соответствует найденной зависимости по сравнению с тем, сколько всего очков. Это указывает, какие диапазоны показывают более сильную зависимость. Стандартная статистика ошибок - это показатели значимости; они подробно обсуждаются в Приложении D.
В некоторых случаях для формирования ячеек можно было использовать более двух атрибутов. Поскольку таблица из 3 или более измерений не может быть отображена, вы можете выбрать, какие два атрибута используются для формирования осей, выбирая их из раскрывающихся списков «Строка заголовка» и «Колонка Заголовок». Поскольку эта двумерная таблица является всего лишь срезом полной n-мерной таблицы, вы также должны выбирать диапазоны для всех атрибутов, которые не отображаются - для этой цели имеется раскрывающийся список для каждого атрибута. Выберите соответствующие диапазоны, и отобразится соответствующий раздел таблицы. Нет эффекта выбора диапазонов для отображаемых атрибутов - для этих атрибутов отображается полный диапазон.
Кроме того, Find Dependencies выводит новое правило для коллекции Rules. Это правило не может быть отображено или отредактировано (оно фактически состоит из таблицы зависимостей), но может применяться к другим наборам данных, чтобы определить, попадают ли записи в зависимую подпопуляцию.